SIMATS ENGINEERING

Engineer to Excel

SIMATS

Saveetha Institute of Medical And Technical Sciences

(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Department of Computer Science and Engineering

Course Code & Name	CSA06 – Design and Analysis of Algorithms
Assessment	CO1 – Assessment Tool 1: IT Application-Based Problem Solving – Scalability Audit of an E-Commerce Platform
Course Outcome Assessed	Determine algorithm efficiency using asymptotic notations and mathematical techniques including Master Theorem and substitution method.
Duration	3 Days
Mode	Individual
Student Reg. No.	192511042
Student Name	Thamaraiselvi S

Instructions: Answer all parts. Show the basic operation, every step of each derivation and the final asymptotic result, and interpret what each result means for the platform. Assume the platform's servers execute 10^8 basic operations per second.

IT APPLICATION-BASED PROBLEM

Scenario – Scalability Audit of "ShopSphere", an Online Shopping Platform: ShopSphere is a growing e-commerce platform whose product catalogue currently holds n = 100,000 products and is expected to reach n = 1,000,000 within a year. Before the expansion, the CTO has asked you, as the platform's algorithm engineer, to audit five back-end modules, predict how each will behave as n grows, and recommend which modules must be redesigned. The modules are described below.

Module M1 – Duplicate-Listing Detector: Sellers sometimes upload the same product twice. Every night, the following routine compares every product with every later product.

```
FOR i ← 0 TO n − 2 DO
   FOR j ← i + 1 TO n − 1 DO
      IF Product[i].title = Product[j].title AND Product[i].seller =
Product[j].seller THEN
         MarkDuplicate(j)
      END IF
   END FOR
END FOR
```

Module M2 – API Rate Limiter: The public API audits request windows of doubling size using the routine below. A team member claims that it is $\Theta(n \log n)$ because the outer loop runs about log n times and the inner loop runs up to n times.

```
i ← 1
WHILE i < n DO
   FOR j ← 0 TO i − 1 DO
      Audit(Request[j])          // basic operation
   END FOR
   i ← i × 2
END WHILE
```

Module M3 – Product Search: Two versions exist. Version S1 scans the unsorted catalogue sequentially until the product is found. Version S2 searches a sorted product index by comparing the key with the middle entry and discarding half the index after each comparison; its comparison count satisfies $T(n) = T(n/2) + 1$, $T(1) = 1$.

Module M4 – Recommendation Engine: The engine splits the customer–product data into four parts of size n/2 (because of replicated partitions) and merges the partial results in linear time: $T(n) = 4T(n/2) + n$. A proposed redesign removes the replication, giving $T(n) = 2T(n/2) + n$.

Module M5 – Sales Analytics Engine: The analytics engine splits the sales data into halves recursively and sorts the records by revenue at every level, giving $T(n) = 2T(n/2) + n \log n$. Separately, profiling shows that the nightly report generator performs $f(n) = 3n^2 + 10n \log_2 n + 50$ operations.

(a) Understand the Problem: For each of the five modules, identify the input size and the basic operation that dominates the running time, explain where it occurs, and state how you expect its execution count to grow as n increases. Justify each choice briefly.

(b) Analyse the Iterative Modules: For M1, derive the exact number of comparisons as a function of n and state the time and space complexity. For M2, derive the exact number of Audit

operations, decide whether the team member's $\Theta(n \log n)$ claim is correct and give the tight bound. Verify your M2 result for n = 1,024.

(c) Solve a Recurrence by Substitution: Solve the recurrence for search version S2 using backward substitution, showing every intermediate step, and derive the exact closed form. State the time complexity and the space complexity of both an iterative and a recursive implementation.

(d) Apply the Master Theorem: For M4, apply the Master Theorem to both the current design and the proposed redesign, clearly identifying a, b, f(n), n^(log_b a) and the case used, and explain what happens to the running time when the data doubles. For M5, determine whether any of the three standard cases applies to $T(n) = 2T(n/2) + n \log n$; if not, derive the complexity using the extended Master Theorem or a recursion tree.

(e) Analyse Best, Average and Worst Cases: For search versions S1 and S2, identify the input condition that produces the best, average and worst case and give the number of comparisons in each. For S1, derive the average number of comparisons when the probability that the searched product exists is p, and evaluate it for n = 1,000,000 and p = 0.5.

(f) Prove an Asymptotic Bound: Using the formal definitions, prove that the report generator's cost $f(n) = 3n^2 + 10n \log_2 n + 50$ is $\Theta(n^2)$ by finding constants c_1, c_2 and n_0, and show that f(n) is not O(n).

(g) Scalability Decision: Estimate the number of basic operations and the running time of M1, M2, S2, M4 (both designs) and M5 for n = 1,000,000. Identify the modules that will fail to meet a nightly processing window of 10 minutes, and recommend, with justification, which modules must be redesigned and how.

Rubrics – Application-Based Question

Criteria	Weightage (%)	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Mark Evaluation:

Criteria	Weightage (%)	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20				
Application of Concepts	30				
Problem-Solving Approach	20				
Accuracy of Solution	20				
Clarity	10				
Total Marks					

Code of Conduct

I **Thamaraiselvi S (192511042)** (Name / Reg. No.) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Thamaraiselvi S
Date: 13.10.2026

CO1 – ASSESSMENT TOOL 1

IT Application-Based Problem Solving – Scalability Audit of an E-Commerce Platform

Objective

To analyse the time and space efficiency of the ShopSphere e-commerce platform algorithms using asymptotic notations, mathematical derivations, the Master Theorem, and substitution/recurrence techniques. The audit considers input sizes from 100,000 to 1,000,000 records and a nightly processing limit of 10 minutes.

Assumptions

- Input size is represented by n.
- The server can perform approximately 10^8 basic operations per second.
- The maximum allowed nightly processing time is 10 minutes = 600 seconds.
- Operation-count estimates are theoretical; actual runtime also depends on implementation, memory, indexing, and hardware.

(a) Identify Input Size, Basic Operation, and Growth Rate

M1 – Pairwise duplicate detector

Input size: n product records. Basic operation: compare a pair of records to check whether they are duplicates. The algorithm checks every unordered pair.

$$C(n) = (n-1) + (n-2) + \cdots + 2 + 1$$

$$C(n) = n(n-1) \,/\, 2 = (n^2 - n) \,/\, 2$$

Therefore, the time complexity is $\Theta(n^2)$.

M2 – Geometric doubling audit loop

Input size: n records. Basic operation: one audit operation during each loop iteration. If the loop variable doubles each time, the number of iterations grows proportionally to n.

$$A(n) = 1 + 2 + 4 + \cdots + 2^k$$

$$A(n) = 2^{k+1} - 1, \text{ where } k = \lfloor \log_2(n-1) \rfloor$$

$$A(n) = 2\text{^}\lfloor \log_2(n-1) \rfloor^{+1} - 1$$

The operation count is $\Theta(n)$, not $\Theta(n \log n)$.

M3 – Search methods

- S1: Linear search checks records one by one. Worst-case time complexity: $\Theta(n)$.
- S2: Binary search repeatedly halves a sorted search range. Worst-case time complexity: $\Theta(\log_2 n)$.

M4 – Divide-and-conquer algorithm

The current recurrence is $T(n) = 4T(n/2) + n$. Its growth rate is $\Theta(n^2)$. A redesigned version with two half-size recursive calls has recurrence $T(n) = 2T(n/2) + n$ and growth rate $\Theta(n \log n)$.

M5 – Divide-and-conquer algorithm with logarithmic extra work

The recurrence is $T(n) = 2T(n/2) + n \log_2 n$. Its growth rate is $\Theta(n \log^2 n)$.

(b) Exact Operation Counts for M1 and M2

M1: Pairwise duplicate detector

For every record, the algorithm compares it with each record that follows it. Thus:

$$C(n) = \Sigma_{i=0}{}^{n-2} (n - i - 1)$$

$$C(n) = (n - 1) + (n - 2) + \cdots + 1$$

$$C(n) = n(n - 1) / 2$$

Time complexity: $\Theta(n^2)$. Auxiliary space: $O(1)$, excluding the input catalogue and any output structure.

$$\text{For } n = 1{,}000{,}000:$$

$$C(n) = 1{,}000{,}000 \times 999{,}999 / 2$$

$$C(n) = 499{,}999{,}500{,}000 \text{ comparisons}$$

This is far too many comparisons for the 10-minute nightly processing window at the assumed server speed.

M2: Geometric doubling loop

For integer $n > 1$, the exact number of iterations for a loop that doubles its value from 1 while it remains less than n is:

$$A(n) = 2\text{^}\lfloor \log_2(n - 1) \rfloor^{+1} - 1$$

$$\text{For } n = 1{,}024\text{: } A(n) = 1{,}023 \text{ operations}$$

Because the operation count is proportional to n, M2 is $\Theta(n)$. The claim that it is $\Theta(n \log n)$ is incorrect.

(c) Recurrence Relation for Binary Search (S2)

Binary search halves the search range in each recursive call and performs constant work outside the recursive call. Its recurrence is:

$$T(n) = T(n/2) + 1, \text{ for } n > 1$$

$$T(1) = 1$$

Expanding the recurrence:

$$T(n) = T(n/2) + 1$$

$$= T(n/4) + 2$$

$$= T(n/8) + 3$$

$$= T(n/2^k) + k$$

Stop when $n/2^k = 1$. Therefore, $2^k = n$ and $k = \log_2 n$.

$$T(n) = T(1) + \log_2 n$$

$$T(n) = \log_2 n + 1, \text{ for powers of } 2$$

Hence, binary search has Θ(log n) time complexity. Iterative binary search uses O(1) auxiliary space; recursive binary search uses O(log n) stack space.

(d) Master Theorem Analysis for M4 and M5

M4 – Current algorithm

$$T(n) = 4T(n/2) + n$$

Here a = 4, b = 2, and f(n) = n.

$$n\text{^}(\log_b\ a) = n\text{^}(\log_2 4) = n^2$$

Since f(n) = n is polynomially smaller than n^2, Master Theorem Case 1 applies.

$$T(n) = \Theta(n^2)$$

M4 – Redesigned algorithm

$$T(n) = 2T(n/2) + n$$

Here a = 2, b = 2, and f(n) = n.

$$n\text{^}(\log_b\ a) = n\text{^}(\log_2 2) = n$$

Since f(n) = Θ(n), Master Theorem Case 2 applies.

$$T(n) = \Theta(n \log n)$$

When n doubles, the current $\Theta(n^2)$ algorithm takes approximately four times as much work. The redesigned Θ(n log n) algorithm takes a little more than twice as much work.

M5 – Extended Master Theorem

$$T(n) = 2T(n/2) + n \log_2 n$$

Here a = 2, b = 2, and n^(log_b a) = n. The additional work is f(n) = n log n, which matches the extended Case 2 form $n\text{^}(\log_b\ a) \log^k n$ with k = 1.

$$T(n) = \Theta(n \log^2 n)$$

(e) Best, Average, and Worst Cases for Search Algorithms

S1 – Linear search

- Best case: 1 comparison, when the target is the first record.
- Average case, assuming the target is present and equally likely to be in any position: (n + 1)/2 comparisons.
- Worst case: n comparisons, when the target is last or absent.

If p is the probability that the target exists, and a present target is equally likely to be in any position, the expected comparison count is:

$$E[C] = p(n + 1)/2 + (1 - p)n$$

$$E[C] = n(1 - p/2) + p/2$$

For n = 1,000,000 and p = 0.5:

$$E[C] = 0.5(1{,}000{,}001)/2 + 0.5(1{,}000{,}000)$$

$$E[C] = 750{,}000.25 \text{ comparisons}$$

S2 – Binary search

- Best case: 1 comparison, when the target is at the middle position.
- Average case: Θ(log n), assuming a sorted array and a suitable distribution of target positions.
- Worst case: Θ(log n), because the search interval is halved at each step.

$$\log_2(1{,}000{,}000) \approx 19.93$$

Binary search therefore needs about 20 comparisons for one search on a sorted dataset. This estimate excludes the cost of creating or maintaining the sorted index.

(f) Prove That $f(n) = 3n^2 + 10n \log_2 n + 50$ Is $\Theta(n^2)$

To prove $f(n) = \Theta(n^2)$, constants $c_1, c_2 > 0$ and n_0 must exist such that $c_1n^2 \leq f(n) \leq c_2n^2$ for all $n \geq n_0$.

Lower bound:

$$f(n) = 3n^2 + 10n \log_2 n + 50 \geq 3n^2$$

Thus, choose $c_1 = 3$.

Upper bound: for $n \geq 1$, $\log_2 n \leq n$ and $50 \leq 50n^2$.

$$f(n) \leq 3n^2 + 10n^2 + 50n^2$$

$$f(n) \leq 63n^2$$

Thus, choose $c_2 = 63$ and $n_0 = 1$. Therefore:

$$3n^2 \leq f(n) \leq 63n^2, \text{ for all } n \geq 1$$

$$\therefore f(n) = \Theta(n^2)$$

Why f(n) is not O(n): if $f(n) \leq cn$ for a fixed constant c, then $3n^2 \leq cn$, which implies $3n \leq c$. This cannot hold for arbitrarily large n. Hence f(n) is not O(n).

(g) Scalability Audit at n = 1,000,000

Assume a server performs 10^8 basic operations per second and the nightly limit is 600 seconds.

Algorithm / task	Estimated operations	Estimated time	Decision
M1: pairwise duplicate detector	499,999,500,000	4,999.995 s ≈ 83.33 min	Fails
M2: doubling audit loop	≈ 1,048,575	≈ 0.0105 s	Passes
S2: one binary search	≈ 20 comparisons	≈ 0.0000002 s	Passes*
M4: current recurrence	$\approx 2 \times 10^{12}$ (illustrative)	≈ 20,000 s ≈ 333.33 min	Fails
M4: redesigned recurrence	$\approx 2.1 \times 10^{7}$ (illustrative)	≈ 0.21 s	Passes
M5 recurrence	$\approx 2.1 \times 10^{8}$ (illustrative)	≈ 2.1 s	Passes
Separate quadratic report generator	$\approx 3.0 \times 10^{12}$	≈ 30,000 s ≈ 500 min	Fails

*The binary-search time shown is for one search only and does not include building or maintaining the sorted index. M4 and M5 figures are illustrative asymptotic estimates; exact counts depend on base cases, constants, and implementation details.

Final Recommendations

- Replace M1 pairwise duplicate detection with sorting followed by adjacent comparisons, or use hash-based grouping when memory allows.
- Keep M2, since its growth is linear and its estimated operation count is small.
- Use S2 binary search only when the data is sorted; include index construction and maintenance costs in the complete system audit.
- Redesign M4 to remove the four recursive calls per level and target $\Theta(n \log n)$ growth.
- Optimize the separately profiled quadratic report generator, because its estimated runtime exceeds the 10-minute nightly window.
- Benchmark the implementation on representative data and monitor runtime, memory use, and indexing overhead before deployment.

Conclusion

The audit shows that algorithmic growth rate is critical to ShopSphere scalability. Quadratic algorithms become impractical at one million records, while linear and logarithmic methods are much more suitable. Replacing inefficient duplicate detection, redesigning M4, and optimizing the report generator will help the platform meet its nightly processing limit.